Relatório Final — Laboratórios DevOps 1, 2 e 3

Este relatório reúne o que você e sua dupla fizeram nos Laboratórios 1, 2 e 3 e mostra que entenderam o porquê de cada etapa. Entrega: **19/10/2026**, aula de fechamento, em um único arquivo PDF.

Identificação

Campo	Preencha aqui
Disciplina	Nuvem e DevOps — Análise e Desenvolvimento de Sistemas (UniEinstein)
Professor	Nelson H Koshoji
A1: Aluno 1 (nome completo e RA)	
A2: Aluno 2 (nome completo e RA)	
Link do repositório no GitHub	
Modalidade	Dupla ou individual (se for sozinho, deixe o Aluno 2 em branco)
Entrega	19/10/2026 (aula de fechamento), pelo Moodle

Como funciona este relatório

Cada laboratório tem três partes. Faça-as na ordem:

1. **Evidências:** links e trechos de código ou saída colados como texto, que provam que vocês executaram o laboratório.
2. **Exercício e cenários:** responda o exercício indicado aqui, que consta no final de cada apostila, e os dois cenários do tipo "o que aconteceria se...".
3. **Questões conceituais:** pesquise fora da apostila e responda com as suas palavras. É aqui que você mostra que entendeu o que o laboratório quis ensinar.

Quem preencher cada resposta escreve ao final dela **A1 (Aluno 1)**, **A2 (Aluno 2)** ou **A1+A2 (Aluno 1 + Aluno 2)**. Cada aluno deve ser o autor de cerca de metade das questões conceituais.

Regras para as respostas

- **Evidências:** cole códigos e saídas de comandos **como texto**, indicando o comando usado. O único print exigido em todo o relatório é o do alerta do Grafana (E3.1). Os links do GitHub devem abrir para o professor.
- **Questões conceituais:** de 4 a 8 linhas, com as suas palavras, e pelo menos **uma fonte confiável** citada (documentação oficial, como docs.docker.com, developer.hashicorp.com/terraform, docs.github.com, prometheus.io/docs ou grafana.com/docs). Texto copiado de um site ou de uma IA sem ter sido entendido vale zero na questão.

- **Se não conseguir responder ou executar algo:** não deixe em branco. Escreva uma justificativa honesta, dizendo o que você tentou, onde travou (cole a mensagem de erro, se houver) e o que pesquisou para tentar resolver. Justificativa específica pode receber crédito parcial, a critério do professor. Item em branco ou com justificativa genérica ("não deu tempo") vale zero.
- **Uso de IA:** pode ser usada como apoio de estudo, conforme o documento "Instruções Gerais". Declare o uso na seção final. Esteja pronto para explicar qualquer resposta oralmente, se o professor pedir.
- **Prazo do Lab 3:** quem ainda não terminou o Lab 3 pode concluí-lo até 19/10. As partes dos Labs 1 e 2 já podem ser preenchidas desde hoje.
- **Arquivo final:** exporte como PDF com o nome `DevOps_Relatorio_Sobrenome1_Sobrenome2.pdf`. Confira se o print do alerta está legível antes de enviar. Entregue pelo Moodle: os dois integrantes devem enviar o arquivo.
- **ATENÇÃO — respostas iguais:** respostas iguais às de outra dupla, ou copiadas de outro relatório, serão **zeradas** para todos os envolvidos, sem distinção de quem copiou de quem. Estudem juntos se quiserem, mas cada dupla deve escrever com as suas próprias palavras.

Laboratório 1 — Infraestrutura como Código (Terraform e Docker)

No Lab 1 vocês provisionaram um servidor web (Nginx) e um banco de dados (PostgreSQL) com Terraform e Docker, e ligaram os dois usando `psql`.

Parte A — Evidências (o que você fez)

E1.1 — Código e resultado. Cole aqui, como texto, o seu `main.tf` do Lab 1 (Nginx, volume, Postgres), o seu `site/index.html`, a saída do `docker ps` com os dois containers e a saída do `curl http://localhost:8080` mostrando a mensagem que veio do banco.

Seu código:

Parte B — Exercício da apostila e cenários (responda em 3 a 6 linhas cada)

Cenário 1 — Estado perdido. Alguém apaga o arquivo `terraform.tfstate` e roda `terraform plan`. O que o Terraform vai propor, mesmo com os containers já rodando, e por que isso é perigoso? Como uma equipe evita esse problema?

Sua resposta:

Cenário 2 — Mudança que recria. Você troca o `name` do container `meu-container-web` no `main.tf` e roda `terraform plan`. O Terraform propõe atualizar no lugar ou destruir e recriar? Por que isso importa muito mais para o banco de dados do que para o Nginx?

Sua resposta:

Exercício 7 — Stateless × stateful. Após destruir e recriar o container do banco, a mensagem continuou lá? Explique o papel do `docker_volume.db_data` e o que aconteceria sem ele. Cole a saída da consulta feita depois de recriar o container.

Sua resposta:

Parte C — Questões conceituais (pesquise e responda com suas palavras)

Responda de 4 a 8 linhas cada, citando ao menos uma fonte. Finalize cada resposta com a autoria (A1, A2 ou A1+A2).

C1.1 — O que é DevOps? Explique como surgiu, que problema resolveu entre as equipes de Desenvolvimento e de Operações e o que significa a sigla **CALMS**. Relacione com a esteira dos três laboratórios (provisionar → automatizar → observar).

Sua resposta e fonte:

C1.2 — Infraestrutura como Código. Defina IaC. Diferencie a abordagem *imperativa* (`docker run nginx`) da *declarativa* (`main.tf`). Cite três benefícios e um risco da IaC. Cite outras duas ferramentas de IaC (por exemplo Ansible, Pulumi, CloudFormation ou OpenTofu) e diga em que diferem do Terraform.

Sua resposta e fonte:

C1.3 — Containers e Docker. Qual a diferença entre container e máquina virtual? E entre *imagem* e *container*? O que é um *registry* (como o Docker Hub)? Por que o Terraform precisou do Docker instalado para funcionar neste lab?

Sua resposta e fonte:

C1.4 — Volumes e serviços com estado. Diferencie *bind mount* (a pasta `site/` do Nginx) de *volume gerenciado* (o volume do Postgres). Explique serviço *stateless* × *stateful* e por que bancos de dados exigem cuidados extras em produção (como backup e réplicas).

Sua resposta e fonte:

C1.5 — Estado, idempotência e drift. O que é o arquivo `terraform.tfstate` e para que serve? Por que ele não deve ser editado à mão nem versionado com senhas? O que é *remote state*? Defina *idempotência* e *drift* e dê um exemplo de cada que você viu no lab.

Sua resposta e fonte:

Laboratório 2 — Automação de deploys com CI/CD (GitHub Actions)

No Lab 2 vocês versionaram o projeto no GitHub e criaram uma pipeline que roda `init → fmt → validate → plan → apply` sozinha a cada mudança.

Parte A — Evidências (o que você fez)

E2.1 — Repositório. Cole o link do repositório no GitHub. Ele deve conter `main.tf`, `site/`, `.gitignore` e `.github/workflows/terraform.yml`, e **não** deve conter a pasta `.terraform/` nem o arquivo `terraform.tfstate`.

Sua evidência:

E2.2 — Execuções da pipeline. Cole três links da aba **Actions**: (1) uma execução **verde**, com `apply`; (2) a execução do Pull Request da branch `feature/nova-porta`, em que o `plan` rodou e o `apply` foi pulado; (3) uma execução **vermelha**, causada pelo `terraform fmt -check`. Ao lado de cada link, escreva em uma linha o que a execução mostra.

Sua evidência:

E2.3 — Teste automatizado do conteúdo (exercício 7). Cole, como texto, o passo *Smoke test* do seu `terraform.yml` (com `curl` e `grep`) e o link da execução em que ele passou.

Sua evidência:

Parte B — Exercício da apostila e cenários (responda em 3 a 6 linhas cada)

Exercício 5 — Rollback. Descreva o incidente que vocês simularam, o `git revert` usado (cole o link do commit de revert) e o que a pipeline fez depois. Que vantagem isso tem sobre corrigir manualmente em produção?

Sua resposta:

Cenário 1 — Erro direto na `main`. Alguém faz `push` direto na `main` com um erro de sintaxe no `main.tf`. Em qual etapa a pipeline falha e o que acontece com a infraestrutura que já estava no ar? Que configuração do GitHub impediria o `push` direto?

Sua resposta:

Cenário 2 — Token vazado. O token de acesso pessoal (PAT) de um integrante é publicado por engano em um commit público. O que alguém mal-intencionado poderia fazer com ele e quais são os primeiros passos para reagir?

Sua resposta:

Parte C — Questões conceituais (pesquise e responda com suas palavras)

Responda de 4 a 8 linhas cada, citando ao menos uma fonte. Finalize cada resposta com a autoria (A1, A2 ou A1+A2).

C2.1 — CI, entrega contínua e implantação contínua. Diferencie *Continuous Integration*, *Continuous Delivery* e *Continuous Deployment*. Qual deles a sua pipeline implementa, e em qual etapa do `terraform.yml` isso fica claro?

Sua resposta e fonte:

C2.2 — Anatomia de uma pipeline no GitHub Actions. Explique os termos *workflow*, *evento (trigger)*, *job*, *step*, *runner* e *action*, apontando onde cada um aparece no seu `terraform.yml`. Por que o runner é efêmero?

Sua resposta e fonte:

C2.3 — Fail fast e controles de qualidade. O que significa *fail fast* e o que é um *quality gate*? Por que o `plan` roda no Pull Request e o `apply` só após o merge na `main`? O que é *branch protection* e como ela reforça esse processo?

Sua resposta e fonte:

C2.4 — Gestão de segredos. Por que nunca se deve versionar senhas e chaves (como o `POSTGRES_PASSWORD` do Lab 1)? Explique como o *GitHub Secrets* funciona e cite duas alternativas usadas na indústria (por exemplo HashiCorp Vault, AWS Secrets Manager ou Azure Key Vault). Qual o papel do `.gitignore` aqui?

Sua resposta e fonte:

C2.5 — Estratégias de deploy, rollback e métricas DORA. Explique *rolling update*, *blue/green* e *canary*, e a diferença entre *rollback* e *roll-forward*. Cite as quatro métricas DORA e diga qual delas a pipeline do Lab 2 melhora mais.

Sua resposta e fonte:

Laboratório 3 — Observabilidade e resposta a incidentes (Prometheus e Grafana)

No Lab 3 vocês acrescentaram cAdvisor, Prometheus e Grafana à infraestrutura, montaram um dashboard de CPU, provocaram um incidente e o documentaram. Quem não terminou pode concluir até 19/10; este é o laboratório com mais itens feitos em casa.

Parte A — Evidências (o que você fez)

E3.1 — Alerta no Grafana (exercício 4). Este é o único print do relatório: a regra de alerta configurada, mostrando consulta, condição e limiar. Se puder, acrescente o alerta em estado *Pending* ou *Firing* durante o teste. Escreva de 3 a 5 linhas justificando o limiar escolhido (lembre que 1 = um núcleo inteiro).

Sua evidência e justificativa:

E3.2 — Mini-postmortem sem culpa (exercício 5 e seção 6 da apostila). Preencha a tabela abaixo sobre o incidente simulado. Seja objetivo e use horários reais.

Item	Sua descrição
O que aconteceu (sintoma observado)	
Quando foi detectado (horário em que o dashboard mostrou o desvio)	
Causa raiz	
Ação tomada para mitigar	
Como evitar ou detectar mais rápido no futuro	

E3.3 — Tudo no GitHub (etapa 5 da apostila). Cole o link do commit "Adiciona stack de observabilidade" e o link da execução verde da pipeline que veio depois dele. O `prometheus.yml` deve estar no repositório. Cole também, como texto, a saída do `docker ps` com os cinco containers: `meu-container-web`, `meu-container-db`, `cadvisor`, `prometheus` e `grafana`.

Sua evidência:

Parte B — Exercício da apostila e cenários (responda em 3 a 6 linhas cada)

Cenário 1 — Prometheus fora do ar. O Prometheus fica 10 minutos parado durante um incidente. O que o dashboard do Grafana mostra para esse intervalo? Esses dados são recuperados depois? Por quê?

Sua resposta:

Exercício 6 — MTTD. Informe quanto tempo passou entre o início do incidente e o momento em que vocês perceberam o pico no dashboard. Proponha uma melhoria que reduziria esse tempo em um cenário real.

Sua resposta (com o tempo medido):

Cenário 2 — Coleta mais frequente. Você reduz o `scrape_interval` de 5s para 1s. O que se ganha na detecção e que custos aparecem (armazenamento, CPU, ruído nos alertas)?

Sua resposta:

Parte C — Questões conceituais (pesquise e responda com suas palavras)

Responda de 4 a 8 linhas cada, citando ao menos uma fonte. Finalize cada resposta com a autoria (A1, A2 ou A1+A2).

C3.1 — Observabilidade × monitoramento. Qual a diferença entre monitorar e observar um sistema? Explique os três pilares (métricas, logs e traces), cite uma ferramenta de cada e diga por que um time costuma começar pelas métricas.

Sua resposta e fonte:

C3.2 — Como o Prometheus funciona. Explique o modelo *pull* (*scrape*) em comparação ao *push* e o que é uma série temporal. Diferencie os tipos de métrica *counter*, *gauge* e *histogram* e diga por que um *counter* precisa do `rate()`. Resuma o papel de cAdvisor, Prometheus e Grafana na cadeia.

Sua resposta e fonte:

C3.3 — Bons alertas, SLI, SLO e SLA. Defina SLI, SLO e SLA e os quatro *golden signals* do livro Google SRE (latência, tráfego, erros e saturação). O que é *alert fatigue* e como um alerta bem escolhido a evita? Relacione com o limiar que você definiu no E3.1.

Sua resposta e fonte:

C3.4 — Resposta a incidentes e postmortem sem culpa. Descreva o ciclo detectar, diagnosticar, mitigar e aprender. Defina MTTD e MTTR. Por que o postmortem deve ser *sem culpa* e o que uma equipe ganha com ele?

Sua resposta e fonte:

C3.5 — Visão integrada e segurança. Explique como IaC, CI/CD e observabilidade formam um ciclo contínuo de melhoria (o que a observabilidade devolve para o desenvolvimento?). Cite o serviço equivalente ao Prometheus e ao Grafana em um provedor de nuvem (AWS, Azure ou GCP). Aponte ainda **dois riscos de segurança** da stack do laboratório (por exemplo cAdvisor com `privileged = true`, Grafana com `admin/admin`, senha do Postgres em texto aberto) e como você os mitigaria em produção.

Sua resposta e fonte:

Reflexão final e declaração de autoria

Reflexão individual (cada aluno escreve a sua, de 5 a 8 linhas)

Responda: qual foi a maior dificuldade que você enfrentou nos três laboratórios e como a resolveu? O que você aprendeu que não sabia antes? O que você faria diferente? Qual foi a sua contribuição pessoal na dupla?

Aluno 1:

Aluno 2:

Declaração de uso de IA

Conforme o documento "Instruções Gerais", informe se usaram ferramentas de IA e para quê (por exemplo: tirar dúvidas, explicar um erro, revisar texto). Se não usaram, escrevam "não utilizamos".

Ferramenta(s) e finalidade:

Declaração de autoria

Declaramos que as respostas deste relatório foram escritas por nós, que as saídas e o print são dos nossos próprios ambientes e que as fontes consultadas foram citadas.

Aluno 1 (nome e data):

Aluno 2 (nome e data):

Como o relatório será avaliado

O relatório vale **10 pontos** e compõe a nota da P1. Cada aluno pode receber nota diferente se a autoria (A1/A2) e a reflexão mostrarem participação desigual.

Critério	Pontos
Evidências dos Labs 1, 2 e 3	1,0
Exercício da apostila e cenários	3,0
Questões conceituais	4,0
Reflexão e declarações	1,0
Organização do PDF	0,5
Equilíbrio entre os integrantes (Autoria A1/A2 distribuída e coerente com as reflexões individuais).	0,5